



IILM UNIVERSITY, GREATER NOIDA

School of Computer Science and Engineering

INTERSHIP PRESENTATION

Object Detection Using Python

Real-Time Object Detection with YOLOv8, OpenCV and Tkinter

STUDENT NAME

Ravikant Mishra

ENROLLMENT / ROLL NUMBER

CS-2341620

PROGRAM & SEMESTER

B.Tech CSE (AI/ML), 7th Semester

ORGANIZATION

EduSkills Academy

Introduction & Background

WHAT IS OBJECT DETECTION

- 1 Identifies objects in an image or video frame AND locates them using bounding boxes — unlike simple classification.
- 2 Applications span autonomous driving, surveillance, facial recognition, medical imaging and smart retail.
- 3 Python's readable syntax and rich ecosystem (OpenCV, deep-learning APIs) make it well suited for this domain.

The project implements a real-time detector using YOLOv8 with OpenCV and a Tkinter GUI — displaying detections, capturing snapshots, and recording annotated video.

PROJECT BACKGROUND

Single-Shot Detectors

One-pass models (e.g. YOLO) predict boxes and classes together for speed.

Transfer Learning

Pre-trained weights are reused instead of training from scratch.

Real-Time Inference

Lightweight models enable detection to run live on video.

Lightweight Models

Nano-sized architectures balance accuracy with speed.

Problem Statement & Objectives

PROBLEM STATEMENT

Develop and implement an efficient, scalable object detection system using Python and state-of-the-art deep learning models and libraries such as Tkinter, Ultralytics and OpenCV.

The system should identify and classify multiple objects in real time with useful accuracy across diverse environments, and remain adaptable to different deployment environments.

KEY OBJECTIVES

- ✓ Develop a Python-based app for real-time video input
- ✓ Use YOLOv8 for object localization and classification
- ✓ Integrate OpenCV for capture, processing & recording
- ✓ Provide a simple Tkinter GUI to operate the detector
- ✓ Display labels and bounding boxes on detections
- ✓ Support snapshot capture and video recording
- ✓ Study limitations and future directions of the field

Scope & Technologies Used

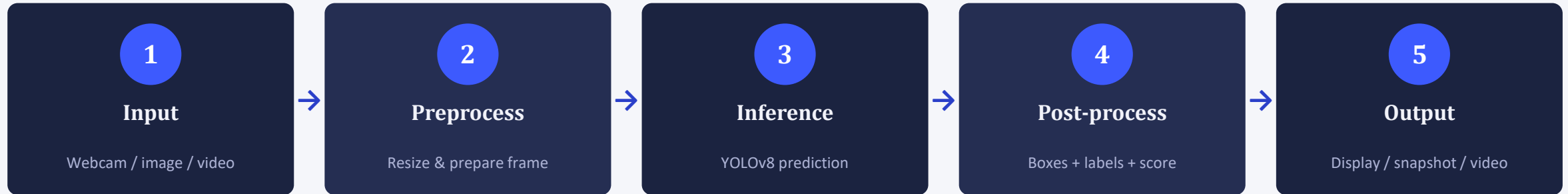
SCOPE OF THE PROJECT

- 1 Real-time webcam-based object detection
- 2 Multiple-object and multi-class detection
- 3 Bounding-box visualization with class labels
- 4 Snapshot and video recording functionality
- 5 A desktop GUI built with Tkinter
- 6 Future extension to edge / mobile deployments

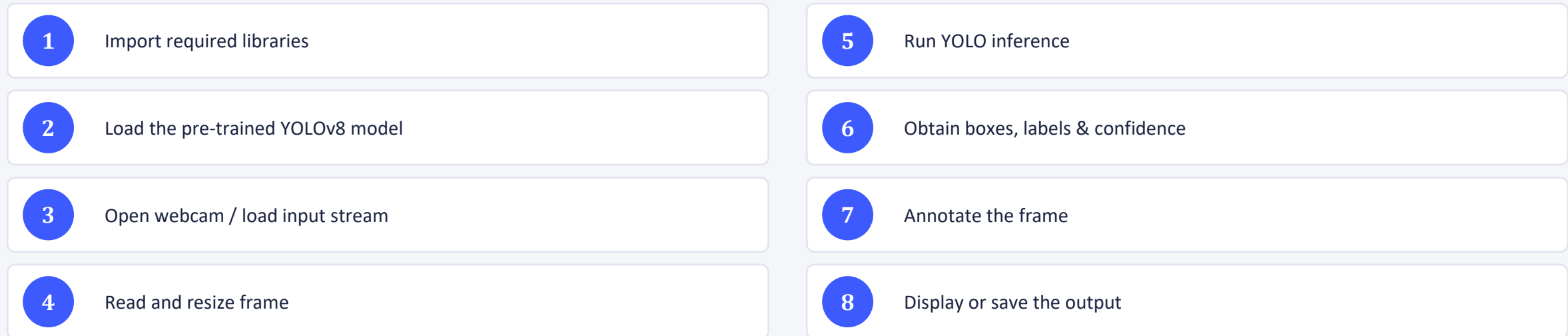
TECHNOLOGIES & TOOLS

Python	Core language — app logic & integration
YOLOv8 / Ultralytics	Object detection model — inference
OpenCV	Capture, resize, colour convert, video I/O
Tkinter	Desktop GUI — interface & controls
Pillow (PIL)	Converts frames for Tkinter display
Threading	Runs detection without blocking GUI
Time	Timestamped snapshot filenames

System Architecture & Detection Pipeline



ALGORITHM STEPS



Methodology

Development followed an iterative build-and-test approach: understanding the problem and selecting a practical pre-trained model, integrating it with OpenCV and Tkinter, then testing the live pipeline.



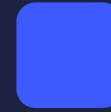
Model & Inference

- Loads the YOLOv8 nano model via Ultralytics
- Each frame resized to 640×480 px
- Result plotted onto the frame as annotation



GUI & Interaction

- Built with Tkinter: title, video area, controls
- Start / Stop / Snapshot / Record buttons
- Background thread keeps GUI responsive



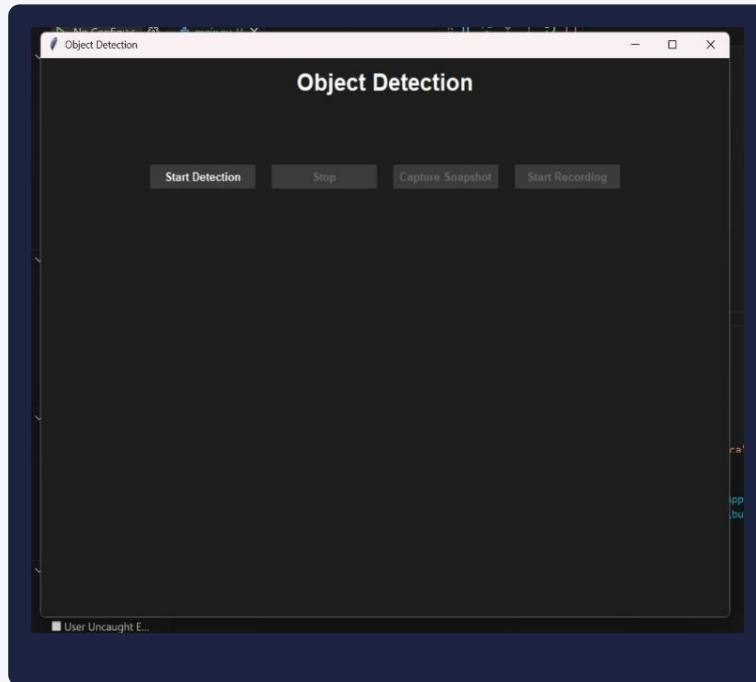
Recording & Snapshot

- VideoWriter uses the XVID codec → AVI file
- Snapshots saved as timestamped JPG
- Latest annotated frame is always cached

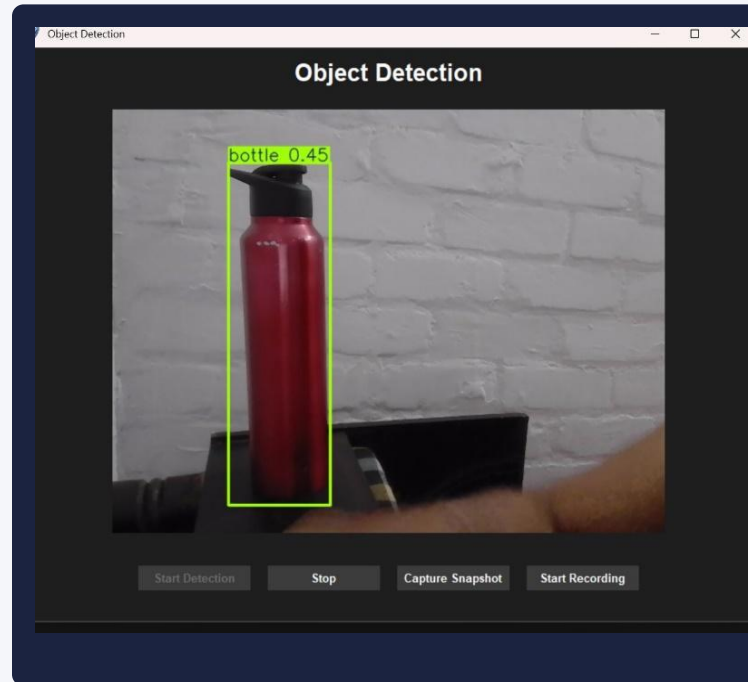
RESULTS

Testing & Demonstration

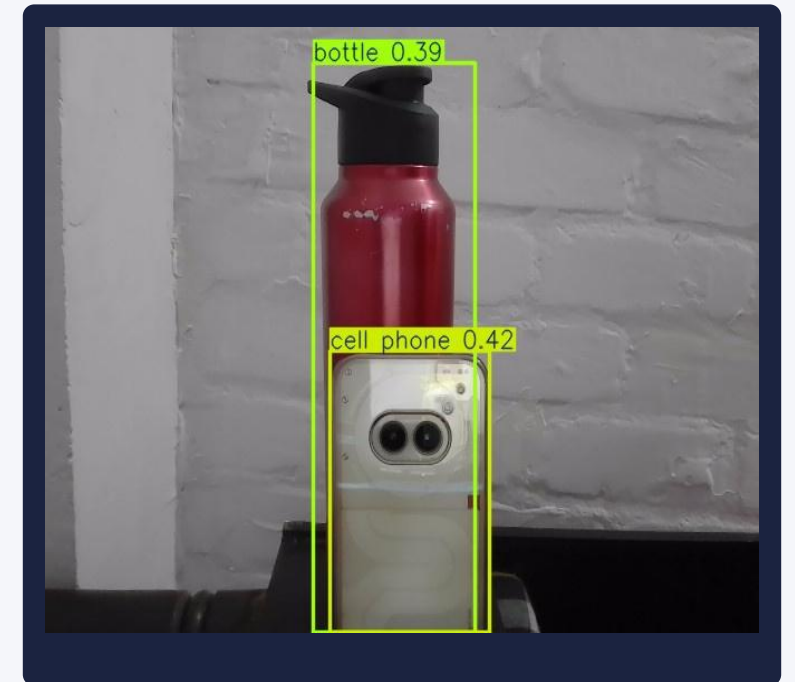
Testing focused on the principal application functions: starting/stopping the webcam, live detection display, recognizing objects in-frame, capturing a snapshot and recording annotated video.



Interface before live detection



YOLOv8 detecting a bottle (0.45 confidence)



Multiple detections — bottle & cell phone

Note: the project source does not provide a formal accuracy benchmark, dataset size, precision/recall table, or FPS measurement — this report does not invent numerical performance results.

Challenges Faced & Expected Outcomes

CHALLENGES FACED

- ! Maintaining real-time performance during per-frame inference
- ! Balancing detection accuracy against computational cost
- ! Handling varied lighting, angles, occlusions & clutter
- ! Keeping the GUI responsive during continuous streaming
- ! Correctly managing video-writer & camera resources

EXPECTED OUTCOMES

- ✓ Real-time object localization and classification
- ✓ Interactive GUI for starting/controlling detection
- ✓ Annotated visual output with boxes and labels
- ✓ Snapshot and recording capabilities
- ✓ A foundation for future deployment & optimization

Future Scope & Conclusion

FUTURE SCOPE

- Higher-accuracy, lower-cost architectures (e.g. DETR)
- Better detection of small / overlapping objects
- Edge & mobile deployment via quantization & pruning
- Integration with augmented-reality applications
- Self-supervised / few-shot learning approaches
- Robustness under poor lighting & dynamic scenes

CONCLUSION

The Object Detection Using Python project demonstrates the practical integration of deep learning and computer vision into a real-time desktop application. Using YOLOv8 through Ultralytics, OpenCV for video processing and Tkinter for the interface, the system detects objects in a live camera feed and displays their locations with bounding boxes and class labels.

While the current implementation is a useful academic prototype, future work can focus on performance benchmarking, custom datasets, model optimization and deployment to mobile or edge platforms.

THANK YOU